Relatório de Testes Unitários

Sistema de Gerenciamento de Biblioteca

Disciplina: Qualidade e Teste de Software
Aluno: José Enzo Marinho Ramos
Docente: Francisco Everson Sousa Leite
Data: 30 de Março de 2026

---

1. Introdução

O presente projeto consiste no desenvolvimento de um **Sistema de Gerenciamento de Biblioteca** utilizando JavaScript puro, com lógica de programação aplicada em camadas (database in-memory, service e controller).

O objetivo principal foi aplicar conhecimentos de **testes unitários** com o framework **Jest**, garantindo a qualidade do código através de pelo menos 10 testes automatizados (foram implementados **12 testes unitários**).

---

2. Elicitação de Requisitos Funcionais

ID	Requisito Funcional	Regra de Negócio / Validação
RF01	Cadastrar livro	ISBN deve ser único, todos os campos obrigatórios, ano válido
RF02	Emprestar livro	Livro deve existir e estar disponível
RF03	Devolver livro	Livro deve existir e estar emprestado
RF04	Buscar livro por ISBN	Retornar o livro ou null
RF05	Listar livros disponíveis	Retornar apenas os livros com status "disponível"
RF06	Calcular multa por atraso	R$ 2,00 por dia de atraso

---

3. Testes Unitários Realizados

Foram criados **12 testes unitários** no arquivo `test/livroService.test.js`, cobrindo caminhos felizes, casos de erro e borda. Todos os testes utilizam `beforeEach` para resetar o banco de dados em memória.

Testes por Requisito:

RF01 – Cadastro de Livro

- Deve cadastrar livro com dados válidos
- Não deve cadastrar livro com ISBN duplicado
- Deve lançar erro ao cadastrar com campos obrigatórios faltando

RF02 – Empréstimo

Deve emprestar livro que está disponível

- Não deve emprestar livro já emprestado

RF03 – Devolução

- Deve devolver livro corretamente (alterar status para disponível)

RF04 – Busca

- Deve encontrar livro por ISBN existente
- Deve retornar null quando o ISBN não existe

RF05 – Listagem

- Deve listar apenas os livros disponíveis

RF06 – Cálculo de Multa

- Deve calcular multa de R$ 0,00 para 0 dias
- Deve calcular multa de R$ 10,00 para 5 dias de atraso
- Deve calcular multa corretamente para 3 dias

Testes adicionais de validação: ano inválido, campos vazios, etc.

---

4. Evidências de Execução

- **Comando executado:** `npm test`
- **Resultado:** 12 testes passaram (12 passed, 0 failed)
- **Cobertura:** Alta cobertura das funções do `livroService.js`

```
PS C:\Users\ramos\OneDrive\Desktop\terceiro ano\Qualidade e Teste de Software\PROJETOS\PROJETO PESSOAL 1> npm test

> projetopessoal-sistema-de-gerenciamento-de-biblioteca@1.0.0 test
> jest

PASS test/livroService.test.js
  LivroService
    √ deve cadastrar livro com dados válidos (8 ms)
    √ não deve cadastrar livro com ISBN duplicado (1 ms)
    √ deve lançar erro ao cadastrar com campos obrigatórios faltando (33 ms)
    √ deve emprestar livro disponível (2 ms)
    √ não deve emprestar livro já emprestado (1 ms)
    √ deve devolver livro emprestado (2 ms)
    √ deve buscar livro por ISBN existente (2 ms)
    √ deve retornar null ao buscar ISBN inexistente (1 ms)
    √ deve listar apenas livros disponíveis (2 ms)
    √ deve calcular multa de R$ 0,00 para 0 dias de atraso (2 ms)
    √ deve calcular multa de R$ 10,00 para 5 dias de atraso (1 ms)
    √ deve calcular multa corretamente para 3 dias (1 ms)

Test Suites: 1 passed, 1 total
Tests:       12 passed, 12 total
Snapshots:   0 total
Time:        1.071 s, estimated 2 s
Ran all test suites.
```

```
PS C:\Users\ramos\OneDrive\Desktop\terceiro ano\Qualidade e Teste de Software\PROJETOS\PROJETO PESSOAL 1> npm start

> projetopessoal-sistema-de-gerenciamento-de-biblioteca@1.0.0 start
> node app.js

==================================================
   SISTEMA DE GERENCIAMENTO DE BIBLIOTECA
==================================================

✅ 3 livros cadastrados com sucesso!

📖 Empréstimo de 'O Senhor dos Anéis': ✅ Sucesso
📚 Livros disponíveis agora: 2

💰 Multa para 8 dias de atraso: R$ 16.00

🎉 Sistema inicializado com sucesso!
   Para rodar os testes: npm test
```

---

5. Tecnologias Utilizadas

- **Linguagem:** JavaScript (Node.js)
- **Framework de testes:** Jest
- **Estrutura:** Camadas (database in-memory, services, controllers)
- **Repositório:** <https://github.com/enzoramoss/projeto-pessoal-sistema-de-gerenciamento-de-biblioteca>

---

6. Conclusão

Através deste projeto foi possível praticar a **elicitação de requisitos funcionais** e a implementação de **testes unitários automatizados** com Jest.

Os 12 testes cobriram as principais regras de negócio do sistema, garantindo que as funções se comportem corretamente tanto em cenários de sucesso quanto em casos de erro.

Essa abordagem demonstra a importância dos testes unitários na garantia da qualidade do software, facilitando a detecção precoce de defeitos e aumentando a confiabilidade do código.

Referência: Projeto baseado na estrutura do Projeto 2 (Sistema de Cadastro de Tarefas).